



Programa de Iniciación Tecnológica

Git y GitHub

Control de Versiones Software

Sesión 2 • Historial, cambios y control del proyecto

Luis Rojas

Instructor del curso • Universidad Nacional de Ingeniería

Modalidad virtual (Zoom + YouTube) • 3 horas con receso

En el aula

- Pregunten sin miedo.
- Equivocarse es parte del oficio.
- Practicamos en vivo.
- Hay receso a mitad de clase (15 a 20 min).

Temas de esta sesión

- Cómo Git registra cambios y arma el historial.
- Modificado, preparado y confirmado.
- **git diff** y **git log** con criterio.
- Recuperar trabajo y escribir commits claros.
- Un repo nuevo con bitácora y varias fotos.

De dónde venimos (sesión 1)

Si la sesión 1 quedó hecha, hoy partimos de esto:

- Git $\neq$ GitHub. Hoy seguimos **en local**.
- Identidad configurada: **user.name** y **user.email**.
- Un repositorio existe: **init**, **add**, **commit**, **log**.
- La brújula sigue siendo **git status**.

Si no alcanzó el laboratorio

Hágalo en los primeros minutos: carpeta, README y un commit. Sirve para mirar **log** y **diff**. En la segunda mitad empezamos un repo nuevo.

Si Git no arranca

git --version. Si falla, reinstale y abra una terminal **nueva**. En Windows: Git Bash.

Al cerrar la sesión 2 deberían poder, con las manos:

- ❶ Explicar qué guarda un commit (instantánea, no un ZIP misterioso).
- ❷ Decir en qué caja está un cambio: trabajo, staging o historial.
- ❸ Leer un **diff** y un **log** antes de confirmar o deshacer.
- ❹ Recuperar un error *según dónde esté*, sin pánico.
- ❺ Tener un repo nuevo (practica-historial) con bitacora.md y varias fotos.

Idea clave. La meta no es memorizar diez comandos. Es reconocer el estado y elegir la acción correcta.

El problema de «guardar» frente a «confirmar»

Guardar en el editor	Preparar (add)	Confirmar (commit)
El archivo queda en disco. Git aún no lo considera historia. Puede perderse con un restore.	Usted elige qué entra en la próxima foto. Es una bandeja, no la caja fuerte.	Aquí nace un punto del historial: autor, fecha, mensaje y hash.

Idea clave. Word «guarda». Git **confirma**. Si no hay commit, no hay versión a la que volver con calma.

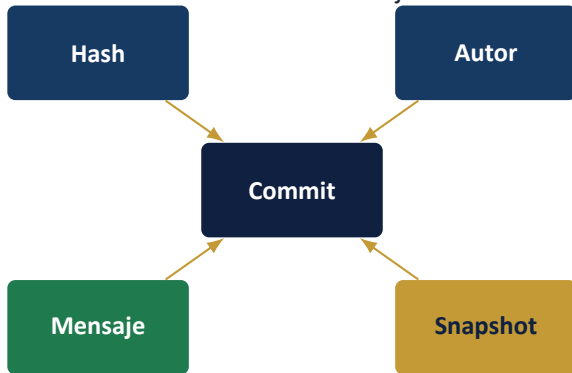
Git no guarda, como pieza central, «la línea 12 cambió de A a B». Guarda una **foto del proyecto** en ese instante.



- Cada commit apunta al anterior (*padre*).
- El hash identifica **esa** foto. Si el contenido cambia, el hash cambia.
- Por eso el historial se puede comparar, explicar y, si hace falta, revertir.

Adaptado de Pro Git, cap. 1.3. No es una cita literal.

Un commit no es «el archivo». Es un objeto con varias piezas:



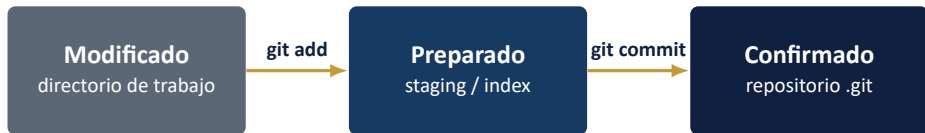
- **Hash:** identificador único (SHA). En el día a día basta el corto: 9f3a21c.
- **Autor y fecha:** la identidad que configuraron en la sesión 1.
- **Mensaje:** el único periódico que van a leer dentro de un mes.
- **Padre:** el commit anterior, salvo el primero del repo.

HEAD es un puntero: «usted está aquí». Casi siempre apunta al último commit de la rama



- **git log** lista el camino hacia atrás desde HEAD.
- **git diff** compara, por defecto, el directorio de trabajo con lo preparado / HEAD.
- Hoy no movemos HEAD a un commit viejo «a ciegas». Primero leemos.

Los tres estados (con criterio)



Taller

Edita, crea, borra. Git lo ve, todavía no lo cuida.

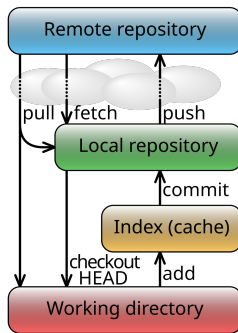
Bandeja

Lo que *elige* meter en la próxima foto. Puede sacarlo.

Caja fuerte

Ya es historia. Se deshace con otro commit, no con goma.

Mapa del flujo local



File:Git data flow simplified.svg · Wikimedia Commons. Working directory → index → HEAD. El recuadro *remote* sigue siendo sesión 4.

```
git status
```

Antes de **diff**, **add**, **commit** o **restore**: status. Responde:

- En qué rama está (hoy suele ser main).
- Archivos *untracked* (nuevos, Git no los cuida).
- Modificados en el taller, aún no preparados.
- Preparados (verdes, si hay color): entran al próximo commit.

Hábito profesional

Status → mire → actúe → status otra vez. El ojo se entrena así, no leyendo un manual.

Ver el cambio antes de la foto: `git diff`

```
git diff  
git diff archivo.md
```

`git diff` muestra lo que cambió en el **directorio de trabajo** y **aún no** está en staging.

- Líneas con + (suele ir en verde): se añadieron.
- Líneas con - (suele ir en rojo): se quitaron.
- El encabezado indica el archivo comparado.

Idea clave. Si el diff no se entiende, el commit tampoco se va a entender. Párese aquí.

git diff --staged: lo que sí entraría

```
git add bitacora.md  
git diff --staged
```

git diff --staged (o **--cached**) muestra lo **ya preparado**: la foto que haría **commit** ahora mismo.

Cuándo usarlo

Después de **add** y antes de **commit**. Es el último vistazo.

Confusión típica

git diff vacío no significa «no hay cambios». Puede significar «todo está en staging». Mire status.

Cómo leer un diff (sin asustarse)

- a/ es el antes; b/ es el después.
- @@ marca el trozo del archivo que cambió.
- No hace falta entender cada símbolo hoy: basta ver *qué* se suma y *qué* se quita.

Mini práctica: leer un diff

En el repo de la demo (si no hay, créenlo en un minuto: **git init** y un README):

```
echo "nota temporal de practica" >> README.md
git status
git diff
git restore README.md
git status
```

Compruebe: el diff muestra la línea nueva; después de **restore**, status vuelve limpio y el archivo ya no tiene esa nota.

Ojo. **git restore** descarta trabajo *no confirmado*. Úselo solo cuando esté seguro de que esa línea no sirve.

git log: la bitácora, no un adorno

```
git log  
git log --oneline
```

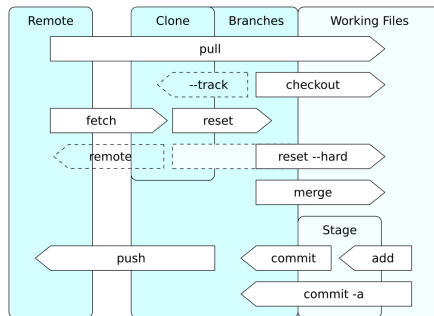
Responde tres preguntas, con evidencia: **qué** (mensaje), **quién y cuándo** (autor y fecha), **cuál foto** (hash). Así se lee **--oneline** en un repo pequeño. Lo repetimos en vivo; si aún no hay historial, un commit basta para verlo:

```
b8c21d4 Agrega lista de comandos  
c4e91ab Agrega README del repositorio
```

--oneline es la vista de todos los días. El log largo, cuando hay que auditar.

Idea clave. Un log corto no es un atraso: es el estado natural al empezar. Después, con varias fotos, volvemos a filtrarlo y a abrir un commit.

Mapa que seguimos recorriendo



File:Git operations.svg · Wikimedia Commons. Hoy profundizamos work / stage / local. **push** y **pull** siguen en la sesión 4.

Pausa

Estírense, tomen agua, despejen la vista del monitor.

Encargo corto durante el receso

- 1 Tenga un repo local y ejecute **git log --oneline**.
- 2 Si no hay repo o el log está vacío, créelo ahora: **git init**, README, un commit. Lo repetimos sin problema.
- 3 Tenga el editor listo: al volver, carpeta nueva y **git init**.

Volvemos para un repo nuevo, commits claros, recuperar cambios y el laboratorio.

Un commit atómico = **un solo cambio lógico**. No «todo lo que toqué esta tarde».

Bien

- Agrega bitacora.md
- Documenta diff y restore
- Ignora logs y archivos .env

Mal

- cambios, fix, asdf, wip
- README + bitácora + .gitignore en un solo commit «porque ya era tarde»

Regla práctica: si no puede decir el cambio en una frase, el commit es demasiado grande.

Mensajes que se pueden leer en un mes

- Modo **imperativo**: *Agrega, Corrige, Actualiza, Ignora*.
- Primera línea corta (cerca de 50–72 caracteres).
- El mensaje describe el *qué*; si hace falta, el cuerpo del commit el *por qué*.
- Hoy usamos `-m` y una frase. El editor largo queda para más adelante.

Documenta diff y restore

update

A la izquierda, una orden al historial. A la derecha, ruido. Dentro de un mes, solo la izquierda sirve.

Hoy un repo nuevo, no el de ayer

Segunda mitad: carpeta nueva, **git init**, un Markdown **completo**. No hace falta haber venido a la sesión 1.

Para qué

- Practicar **diff**, **add** y commits atómicos sobre un archivo real.
- Dejar varias fotos en el log, no un solo commit.
- Llegar a GitHub (sesión 4) con historia, no con una carpeta vacía.

Qué van a crear

- Carpeta: `practica-historia1`.
- Archivo: `bitacora.md`, con frases que se puedan leer en voz alta.
- Dos commits sobre ese archivo, no uno.

Idea clave. Si el archivo no se puede leer en voz alta, el commit tampoco va a servir de bitácora.

Cree bitacora.md *así de lleno* (cambien los datos, no dejen huecos):

```
# Bitacora
- Nombre: Ana Perez
- Sistema: Windows + Git Bash
- Repo: practica-historial

## Inicio
- Repo nuevo con git init.
- Git guarda fotos, no un guardar de word.

## Oficio de hoy
- git diff mira el taller; git diff --staged mira la bandeja.
- restore descarta lo no confirmado.

- Un commit = un cambio logico.
```

Cómo se usa: un git init nuevo

Carpeta nueva. Primer commit: nace el archivo (Inicio). Segundo: se añade Oficio de hoy.

```
mkdir practica-historial
cd practica-historial
git init
git add bitacora.md
git commit -m "Agrega bitacora.md"
git add bitacora.md
git commit -m "Documenta diff y restore"
git log --oneline
```

Cree el archivo en el editor *antes* del primer **add**. Entre los dos commits, edite y añada la segunda sección.

Ojo. Si dejan la plantilla vacía («Nombre:», «Oficio:»), el historial no enseña nada. El ejemplo tiene que leerse.

Primero: ¿dónde está el cambio?

No todos los errores se corrigen igual. Ubique la caja y *después* elija el comando.

Aún no hay commit

- En el taller: **git restore archivo**
- En la bandeja: **git restore --staged archivo**

El segundo conserva el trabajo; el primero lo descarta del taller.

Ya hay commit

- Último y solo suyo: **git commit --amend**
- Hay que deshacer sin borrar: **git revert hash**

Revert crea un commit nuevo. El error original sigue visible.

Ojo. Antes de deshacer: **git status** y **git diff**. Si el trabajo no está en un commit, **restore** puede borrarlo del taller.

git restore: descartar en el taller

Cambió un archivo, no hizo **add**, y esa idea no sirve:

```
git diff
git restore bitacora.md
git status
```

- Vuelve el archivo a la última versión **confirmada** (HEAD).
- Los cambios no preparados **desaparecen** de ese archivo.
- En documentación vieja verán **git checkout -- archivo**. Hoy usamos **restore**.

git restore --staged: sacar de la bandeja

Hizo **add** por error o quiere un commit más chico:

```
git add .  
git status  
git restore --staged comandos.txt  
git status
```

- El archivo sale del próximo commit.
- **Los cambios siguen** en el directorio de trabajo.
- Luego puede confirmar solo `bi tacora.md` y dejar el otro para después.

Para qué sirve staging, entonces

Para no meter el almuerzo y el arreglo del login en la misma foto.

Corregir el último commit: amend

Sirve para un mensaje malo o un archivo olvidado **en el último commit**, si todavía es solo suyo.

```
git commit --amend -m "Documenta diff y restore"
```

O, si faltó un archivo:

```
git add bitacora.md  
git commit --amend --no-edit
```

- **amend reescribe** el último commit (el hash cambia).
- Hoy es seguro: nadie más tiene ese historial.

Ojo. Cuando exista GitHub y otras personas, no reescriba commits ya publicados. Ahí se usa **revert**.

git revert: deshacer sin borrar historia

Si un commit ya no debe aplicar, no lo borramos: creamos un commit que lo deshace.

```
git log --oneline  
git revert HEAD  
git log --oneline
```

- **HEAD** es el último commit. También puede usar un hash corto.
- Git abre editor o pide mensaje; el predeterminado suele bastar.
- El commit original **sigue** en el log. Eso es una virtud, no un fallo.

Idea clave. Revertir es honesto: el error existió y se corrigió a la vista de todos.

Lo que no hacemos hoy (a propósito)

Para no mezclar oficios de rescate:

- No usamos **git reset --hard**. Puede borrar trabajo no confirmado **sin diálogo**.
- No reescribimos historia antigua ni «viajamos» a un commit suelto.
- No hay **stash**, **rebase** ni **cherry-pick**.
- No hay **push**: `bitacora.md` todavía es local.

```
# no ejecute esto en el laboratorio de hoy  
# git reset --hard HEAD
```

Si un tutorial de internet lo pide y usted no tiene commit de respaldo, cierre la pestaña y pregunte.

Ignorar lo que no debe versionarse

`.gitignore` le dice a Git qué **no** rastrear. Los ignorados no salen en status ni entran con `add ..`

```
.env  
*.log  
*.tmp  
__pycache__/  
node_modules/
```

- Secretos: `.env`, claves, tokens. Nunca al historial.
- Basura: logs, temporales, carpetas generadas.
- El propio `.gitignore` **sí** se confirma: es parte del proyecto.

Comprobar que ignora de verdad

```
echo "TOKEN=no-subir" > .env  
echo "ruido" > practica.log  
git status
```

Si `.gitignore` está bien, `.env` y `practica.log` **no** aparecen. Deberían verse `.gitignore` y los archivos que sí importan.

Ojo. Si el secreto ya se confirmó una vez, ignorarlo después *no* lo borra del historial. Hoy no lo subimos. En GitHub sería tarde.

Objetivo: un repo nuevo, `bitacora.md` completo y un rescate consciente.

- ① `mkdir practica-historial`, `cd`, `git init`. `git status`.
- ② Cree `bitacora.md` (Inicio). `add` → commit: Agrega `bitacora.md`.
- ③ Añada Oficio de hoy. Segundo commit: Documenta `diff` y `restore`.
- ④ Escriba una línea basura en `bitacora.md`, mire `diff` y descártela con `restore`.
- ⑤ Cree `.gitignore` (`.env`, `*.log`), un `.env` de prueba y un commit: Ignora secretos y logs.
- ⑥ Deje el `git log --oneline` a la vista: ahora sí hay varias fotos para leer.

- **git diff vacío** con cambios en status verdes: mire **git diff --staged**.
- **Commit sin add.** *nothing added to commit*. Falta staging.
- **restore del archivo bueno.** Status y diff *antes*. Restore no pide confirmación amable.
- **Un solo commit con bitácora + gitignore.** Separe por intención.
- **Mensaje update.** Enmiéndelo ahora con **amend**: todavía es local.

Si algo falla: **pwd, ls -la, git status**. Siguen resolviendo el 80 % de los sustos.

Ahora sí hay historial que mirar

Antes del receso el log era corto (uno o dos commits). Después del laboratorio debería parecerse a esto:

```
c8d21aa Ignora secretos y logs  
7b4e019 Documenta diff y restore  
4a91c33 Agrega bitacora.md
```

- Cada línea es **una foto**: hash corto + mensaje.
- Los hashes de la diapositiva son de ejemplo. Los suyos serán otros.
- Si todavía ven una sola línea, falta un commit del laboratorio: háganlo antes de filtrar.

Idea clave. Estos comandos no se entienden con un solo commit. Por eso los dejamos para ahora.

Recortar y filtrar: -3 y una ruta

```
git log -3  
git log --oneline -- bitacora.md
```

```
git log -3
```

Solo los **últimos tres** commits. En un repo largo no quieren recorrer doscientos. El número es un tope, no «la sesión 3».

Ruta al final

-- **bitacora.md** pide el historial **de ese archivo**. El -- separa opciones de la ruta. La bitácora puede tener dos fotos; .gitignore, una.

```
# git log -3 --oneline          -> las 3 fotos de arriba  
# git log --oneline -- bitacora.md  
7b4e019 Documenta diff y restore  
4a91c33 Agrega bitacora.md
```

--graph --all: el dibujo de las líneas de tiempo

```
git log --oneline --graph --all
```

```
* c8d21aa (HEAD -> main) Ignora secretos y logs  
* 7b4e019 Documenta diff y restore  
* 4a91c33 Agrega bitacora.md
```

- **--graph**: dibuja la línea de tiempo (asteriscos y rayas).
- **--all**: mira **todas** las ramas, no solo en la que están.
- Hoy casi siempre es **una sola línea**: no hay ramas nuevas todavía. En la sesión 3 el dibujo se parte.

git show: abrir una foto

```
git show HEAD  
git show 7b4e019
```

git show abre **un** commit: mensaje + el diff de esa foto contra su padre.

git show HEAD

HEAD = «usted está aquí». Abre el último commit de la rama, el de arriba del log.

git show + hash

Copia el hash **de su log**, no el de la diapositiva. 7b4e019 aquí es un ejemplo. q cierra el pager.

Ojo. Un tutorial con `git show 9f3a21c` no funciona en su repo: ese hash no existe. El hash identifica *su* foto.

Qué no haremos todavía (a propósito)

- No creamos cuenta de GitHub ni hacemos **git push**.
- No clonamos, no hay fork, no hay Pull Request.
- No creamos ramas nuevas ni resolvemos conflictos (sesión 3).
- No usamos reset duro ni reescribimos historia publicada.

Hoy el oficio es: **ver**, **elegir** y **confirmar** sin miedo. Publicar llega cuando sepan qué están publicando.

- ① Git guarda instantáneas. Un commit tiene hash, autor, mensaje y padre.
- ② Tres cajas: taller, bandeja, caja fuerte. Status dice en cuál está el cambio.
- ③ **diff** mira el taller; **diff --staged** mira la bandeja; **log** y **show** leen historia.
- ④ Recuperar: **restore**, **restore --staged**, **amend** (último y local), **revert** (sin borrar).
- ⑤ **.gitignore** deja fuera secretos y basura.
- ⑥ **bitacora.md** nació hoy, con commits atómicos y un log que ya se puede filtrar.

Hoja de comandos de hoy (guárdela)

- 1 Deje `practica-historial` con `bitacora.md` completo (sin huecos) y un `log --oneline` con varias fotos.
- 2 Practique `git log -3`, `git log --oneline -- bitacora.md` y `git show HEAD`.
- 3 Complete el laboratorio si no alcanzó en clase.
- 4 Lea Pro Git en español, cap. 2 (secciones de registro de cambios y de deshacer):
<https://git-scm.com/book/es/v2/Fundamentos-de-Git-Registrando-cambios-en-el-repositorio>
- 5 Traiga una duda escrita sobre *un* comando de hoy, no un «no me funciona» suelto.

Próxima sesión: ramas, **switch**, **merge** y conflictos guiados. Ese repo sigue con ustedes.

- Documentación oficial: <https://git-scm.com/doc>
- *Pro Git* (Chacon y Straub), cap. 2: <https://git-scm.com/book/es/v2>
- **git diff / restore / revert**: páginas git-diff, git-restore, git-revert en <https://git-scm.com>
- Material de apoyo del curso:
<https://github.com/purpesec/curso-git-github-desde-cero>
- Descarga de Git: <https://git-scm.com/downloads>

Las diapositivas son material original del curso PIT 2026. Los conceptos se contrastaron con Pro Git y con el programa PIT686; las imágenes tienen ficha en `imagenes/ATRIBUCION.txt`.

Gracias.

Luis Rojas

Sesión 2 · PIT 2026 · Universidad Nacional de Ingeniería

¿Preguntas de último minuto? El chat y el aula virtual siguen abiertos.